Pigeonhole-Based Seeding for Sequence-to-Graph Alignment on Pangenome Graphs

Carlo Briscolini
Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico di Milano
Milano, Italy
carlo.briscolini@mail.polimi.it
https://github.com/carlobriscolini/GIN-TONIC_pigeonhole

***Abstract*—Linear reference genomes cannot capture the variation present in a population. Pangenome graphs address this by encoding many sequences in a single labelled directed graph, where each path is one possible variant, but most short-read aligners are still built for linear references. We extend GIN-TONIC, an FM-index for graph genomes, with a new pigeonhole subcommand that performs short-read seeding and verification directly on the graph. The seeder walks each read right-to-left, growing a candidate region one character at a time and committing it as a seed once its exact graph occurrences fall below a user threshold. For each surviving seed we decode its graph coordinate, extract a bounded local subgraph via bidirectional BFS, and align the full read against it with Theseus under affine-gap penalties. A small window of nearby start offsets is tried around each seed to absorb indels in the part of the read before the seed.**

I. Introduction

Next-generation sequencing has made genomic data available at a scale that exposes the limits of a single linear reference. A linear reference stores a single version of the sequence at every position in the genome, so it cannot represent the variation found across a population. Sequences that differ from the chosen reference are systematically underrepresented, an effect known as reference allele bias [1]. The bias carries into read mapping: SNPs, short indels and larger structural variants present in other individuals are missing from the reference, so reads that cover them align poorly or do not align at all. Mapping each genome against a pangenome reference instead of a single sequence reduces this bias and recovers a broader range of genetic diversity [2].

Pangenome graphs [3], [4] replace the single reference with a labelled directed graph: vertices carry DNA sequence, edges connect adjacent stretches, and a walk through the graph spells out one possible sequence. Reads can then be aligned against any walk, not against a fixed string. Graph aligners still use seed-and-extend. An index returns short exact matches; an approximate aligner then checks whether the full read fits the graph around each seed. Sensitivity is what makes the seeding step useful: a read that differs from its origin path by at most k edits must produce at least one error-free seed.

On linear references, the pigeonhole principle gives this for free. Cut R into $k{+}1$ contiguous pieces; at most k of them contain an error, so at least one is exact and an FM-index finds it [5], [6]. The same argument carries over to graphs once the index can match walks in G, which is what GIN-TONIC [7] provides.

This paper plugs a pigeonhole seeder into GIN-TONIC and connects it to the Theseus graph aligner [10] for verification (Fig. 1). The seeding loop walks each read right-to-left, growing a candidate region one character at a time and committing it as a seed once its exact graph occurrences fall below a user-chosen threshold τ. This trades the strict $k{+}1$ partition for fewer, more selective seeds. For each surviving fork we resolve its suffix-array interval into one or more concrete graph positions (v, off), extract a bounded local subgraph around each position via bidirectional BFS over a CSR view of the graph, and align the full read against it with Theseus;

The implementation is added as a new `gin pigeonhole` subcommand that runs reads in batches and parallelises seeding and verification with OpenMP.

II. Background

A. GIN-TONIC

GIN-TONIC [7] constructs a generalised FM-index over a string-labelled directed graph $G = (V, E)$ by concatenating all vertex labels into a single text T separated by sentinel characters that delimit node boundaries. The Burrows-Wheeler Transform (BWT) of T is augmented with rank and suffix-array (SA) samples, exposing constant-amortised-time character-rank queries and on-demand offset reconstruction. Standard FM-index backward search alone, however, is unaware of the graph adjacency: a query reaching the first character of a node label has no way to continue into the preceding node along an edge of E. GIN-TONIC closes this gap with an *interval-merge tree* (IMT) that records, for every SA interval whose suffixes begin at a node start, the union of SA intervals belonging to the predecessor nodes in G. Backward search proceeds character by character from the end of a query via the LF-mapping; whenever the search reaches the start of a node, the IMT is consulted to instantiate one new *fork node* for each predecessor, allowing matching to continue across node boundaries as if the predecessor label had been concatenated. Each surviving fork at the end of the search represents a contiguous exact-match location (v, off) in the graph. An optional precomputed cache stores SA intervals for

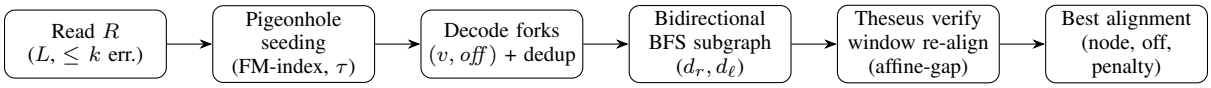


Fig. 1. The two-stage pipeline. A read is partitioned into seed regions by a sliding window over the GIN-TONIC FM-index; each fork is decoded to a graph position, deduplicated, and surrounded by a bounded local subgraph via bidirectional BFS; Theseus then verifies the full read against the subgraph under affine-gap penalties, with a small window of start offsets tried to absorb errors near the seed.

all k-mers up to a configurable depth, replacing the first k backward-search steps of every query with a single lookup; in our deployment the cache file is parsed once at startup and shared across all queries in the batch.

B. Pigeonhole Principle for Seeding

Given read R of length L with error budget k, partition R into $k{+}1$ contiguous segments $s_0, \ldots, s_k$. Since at most k segments can each contain at least one error, there exists an index i such that s_i is error-free. Finding s_i in the index yields a *seed* whose graph position anchors the downstream alignment. The most common instantiation uses fixed-length segments of size $\lfloor L/(k{+}1) \rfloor$, which gives the strongest formal guarantee but produces many short, ambiguous seeds whose occurrences scale linearly with the graph size.

A more selective alternative is to choose segment lengths adaptively. Starting from the right end of the read and walking leftward, the candidate segment is grown one character at a time and its occurrence count is queried at every step; the segment is committed as a seed as soon as the count is at most a threshold τ chosen by the user (Algorithm 1). The resulting partition consists of variable-length, mutually disjoint segments whose specificity is bounded by τ, at the cost of weakening the strict $k{+}1$-segment guarantee when the partition happens to yield fewer than $k{+}1$ regions. In practice the dynamic scheme trades a degree of theoretical worst-case sensitivity for a bound on the candidates each seed produces: every committed region has at most τ exact occurrences in the graph, which directly caps the work performed in the verification stage.

C. Theseus Aligner

Theseus [10] is an output-sensitive sequence-to-graph aligner that computes the *optimal* affine-gap alignment of a read against an arbitrary directed graph, including graphs that contain cycles. Rather than filling the full $O(m \cdot L)$ dynamic-programming matrices used by classical formulations, it generalises the Wavefront Alignment Algorithm (WFA) [9] to graph references: wavefronts of furthest-reaching cells are expanded in increasing order of score s, and the search terminates at the first complete alignment of the query, whose score is therefore the optimum s^*. Efficiency on graphs rests on three ingredients: *sparse wavefronts* that store only the diagonals actually reached at each score, a *Unified Coordinate System* that combines wavefronts arriving at a vertex from its predecessors, and a *diagonal-invalidation strategy* that suppresses redundant exploration under both explicit and gap-propagated extensions. A Singletrack-inspired backtrace then recovers the optimal alignment path from the match-state values alone, keeping the memory footprint low. Given an explicit subgraph and a start position (v_0, off_0), Theseus exposes a C++ API through `TheseusAlignerImpl` and returns the optimal alignment together with its affine-gap score under the penalty tuple (a, x, o, e).

D. Related Work

Most production pangenome read mappers follow the same two-stage template: locate candidate positions through a seeding index, then verify each candidate with a sequence-to-graph aligner. The `vg` toolkit [3] seeds by extracting Maximal Exact Matches (MEMs) from a GCSA2 index built over a variation graph, clusters MEM hits to induce candidate subgraphs, and aligns each subgraph with an affine-gap dynamic-programming kernel. GraphAligner [8] indexes minimizers, chains them with a banded extension, and supports both short and long reads. These mature mappers include substantial engineering beyond the scope of this project, including seed clustering, chaining, mapping-quality estimation, and production-grade output formats. The implementation studied here isolates a narrower question: whether a GIN-TONIC exact graph index can drive a pigeonhole-style filter whose candidate positions are then verified by an optimal sequence-to-graph aligner. This makes the seeding guarantee, verification cost, and implementation trade-offs easier to inspect independently of the full mapper stack.

For linear references, the pigeonhole strategy is the foundation of the GEM mapper [5]. Carrying the same argument into graph references requires only that the index can answer exact-match queries over walks in G, which is precisely the capability provided by GIN-TONIC [7]. Our work occupies a niche that none of the above tools cover: it pairs a pigeonhole-style FM-index seeder operating natively on a pangenome graph with Theseus, an exact aligner that supports the full class of directed graphs without DAG-isation.

III. Implementation

The pigeonhole functionality is exposed as a new `pigeonhole` subcommand of the `gin` binary, registered alongside the existing `index`, `query` and `decode` modes. It accepts a pre-built `.gini` index (`-r`), a required graph file in the `.ging` line format (`-g`), input queries (`-i`), an optional cache (`-C`), a match-count threshold (`-t`, default 1), a batch size (`-b`, default 8), and a thread count (`-j`, default 1). The current implementation writes alignments and final statistics to `alignment_log.txt`. The decoder is always initialised because verification needs to translate suffix-array intervals into graph coordinates; therefore no separate `--decode` flag is exposed by this mode. At startup the index is loaded

Algorithm 1 PigeonholeSeed

Require: Read R (length L), GIN-TONIC index $\mathcal{G}$, threshold τ
Ensure: List of regions $\mathcal{R}$

$\mathcal{R} \leftarrow \emptyset$; $e \leftarrow L$ ▷ right boundary
for $\ell \leftarrow L-1$ **downto** 0 **do**
  $s \leftarrow R[\ell \,..\, e)$
  $(forks, n) \leftarrow \mathcal{G}.\text{QUERY}(s)$
  if $n \le \tau$ **then**
    $\mathcal{R}.\text{APPEND}(\langle \ell, s, forks \rangle)$
    $e \leftarrow \ell$ ▷ slide window
  end if
end for
return $\mathcal{R}$

into a contiguous byte buffer, and the graph is parsed and converted into a compressed-sparse-row (CSR) representation (`gin_csr_graph_t`) that stores per-node label lengths and forward and backward edge arrays for fast neighbour lookup.

A. Seeding Phase

For each read R of length L, seeds are extracted by a right-to-left sliding-window procedure. A right boundary pointer `local_end` is initialised to L. For each region, `local_start` is moved leftward until the current substring has at most τ exact graph occurrences. Without a cache, the first one-character suffix is initialised as a root fork and then extended one character at a time through GIN-TONIC backward search. With a cache, the first tested candidate for a region is the suffix of length $\min($`gin_cache->depth`$,$ `local_end`$)$, obtained with `gin_gin_cache_lookup`; shorter suffixes below the cache depth are not tested in this mode, and only the remaining characters to the left are extended one at a time when the candidate read from the cache still has more than τ occurrences. The threshold test sits inside the per-character extension loop: each candidate is grown by a single LF-mapping step on the existing fork vector, the new occurrence count is checked, and if it is still above τ the next character is appended without restarting the backward search.

The occurrence count is computed as the sum of the SA-interval widths `sa_hi`$-$`sa_lo` across all returned fork nodes. If the count is at most τ, the substring qualifies as a seed: it is saved as a `query_region_t` record carrying the query offset $q_o =$ `local_start`, the seed string, and the vector of exact-match forks. After the record is appended to the per-read region list, `local_end` is reassigned to `local_start`, and the next region starts immediately to the left of the just-committed seed. If a candidate reaches the beginning of the read while still exceeding τ, it is discarded and the partition stops, matching the current implementation's choice to avoid emitting an overly ambiguous final region.

Reads are processed in batches of configurable size (parameter `-b`, default 8); one batch at a time is read into an array of `query_task_t` entries, each holding the read string and its discovered regions. The reads within a batch are independent, so both the seeding and the verification loops iterate over the array and process one read per iteration. When the project is compiled with `BUILD_OPENMP` these iterations are distributed across threads via `#pragma omp parallel for`, so reads of the same batch are seeded and verified concurrently; otherwise the same code path runs serially. The FM-index, cache, CSR graph and decoder are shared read-only, while each verification task owns its subgraph, visited array and deduplication table, and writes to the alignment log inside a critical section.

B. Local Subgraph Extraction

Each fork node in a region is decoded to a concrete graph location $(v_{\text{seed}}, \mathit{off}_{\text{seed}})$ via the GIN-TONIC decoder. A 64-bit deduplication key

$$k_{\text{dup}} = (v_{\text{seed}} \ll 32) \mid \text{low}_{32}(\mathit{off}_{\text{seed}} - q_o) \tag{1}$$

is computed for each decoded position; the upper 32 bits hold the seed vertex id, while the lower 32 bits hold the inferred read start offset $\mathit{off}_{\text{seed}} - q_o$. The offset may be negative when the inferred start falls on a predecessor of the seed vertex; in that case the implementation casts the signed value to `uint32_t` and stores its two's-complement bit pattern, so every distinct $(v_{\text{seed}}, \mathit{off}_{\text{seed}} - q_o)$ pair produces a distinct key without collisions. Positions that hash to a key already seen in the current read are skipped to avoid redundant alignment calls.

For each unique position, the local subgraph is extracted by a forward and a backward BFS from the seed (Algorithm 2). Let $q_r = L - (q_o + |s|)$ be the query bases to the right of the seed and $q_\ell = q_o$ those to the left. The exploration radii are

$$d_r = q_r + k, \qquad d_\ell = q_\ell + k, \tag{2}$$

where k is the maximum error budget ($k = 5$ in the current build). The additive k terms accommodate the worst case in which all errors are deletions in the read (equivalently, insertions in the graph) concentrated on one side of the seed: every such deletion corresponds to a graph base that has no counterpart in the query, so the alignment path must extend k additional graph bases beyond the strict q_r (or q_ℓ) length. Read insertions consume read bases without advancing along the graph, and mismatches consume one base on each side, so neither extends the path beyond $q_r + k$. The radius is therefore safe in both directions for any combination of the three error types.

Both passes operate on a single *visited* array of one byte per graph vertex, initialised before the right-side BFS and reused for the left-side BFS. This shared structure serves a dual purpose. First, it deduplicates the node list so that vertices reachable from both sides of the seed are emitted exactly once. Second, it cuts off cycles in either pass without any special-case logic, which matters because the underlying pangenome graph may contain non-trivial cycles. The seed vertex v_{seed} is treated specially during base consumption: expanding rightward it contributes $|v_{\text{seed}}.\mathit{label}| - (\mathit{off}_{\text{seed}} + |s|)$

Algorithm 2 BidirectionalBFS

Require: Graph G, seed (v, off), seed length $|s|$, radii d_r, d_ℓ
Ensure: Subgraph node set S

$S \leftarrow \{v\}$; $\mathcal{H} \leftarrow \{v\}$ ▷ visited
$Q \leftarrow [(v, 0)]$ ▷ right expansion
while $Q \neq \emptyset$ **do**
  $(u, d) \leftarrow Q.\text{DEQUEUE}()$
  if $d \geq d_r$ **then continue**
  end if
  $c \leftarrow |u.\mathit{label}|$
  if $u{=}v$ **then**
    $c \leftarrow c - (\mathit{off} + |s|)$
  end if
  for all $w \in \text{succ}(u),\ w \notin \mathcal{H}$ **do**
    $S \leftarrow S \cup \{w\}$; $\mathcal{H} \leftarrow \mathcal{H} \cup \{w\}$
    $Q.\text{ENQUEUE}(w, d + c)$
  end for
end while
repeat using $\text{pred}(\cdot)$, radius d_ℓ, and seed offset off ▷ left
return S

bases (the suffix following the seed); expanding leftward it contributes $\mathit{off}_{\text{seed}}$ bases (the prefix before the seed). Successor or predecessor vertices contribute their full label length. The subgraph is stored as a flat vector of vertex IDs in insertion order.

C. Alignment Verification

The node and edge sets of the local subgraph S are converted to arrays of `th_node_t` and `th_edge_t` structures by `build_theseus_data`. Each `th_node_t` carries the vertex ID, the sequence label, and the label length; each `th_edge_t` holds a source/destination pair. Edges are produced by scanning the CSR out-edges of the subgraph nodes and keeping those whose destination is also in S.

The verification loop visits every region, fork and decoded seed hit of a read. An alignment call is made for each unique effective read placement, but the same physical graph position can be reached from independent regions or from different forks of the same region. To avoid redundant alignment calls, a per-read deduplication table `processed_origins` maps the 64-bit key of (1) to a sentinel value; a hit on this table causes the current decoded match to be skipped before any BFS work is initiated. The current command keeps the decoder limit internal and passes `max_matches=-1`, so all matches represented by a selected fork are decoded.

Before invoking the aligner, the read start point (v_0, off_0) must be chosen. Let $o = \mathit{off}_{\text{seed}} - q_o$ be the implied start offset inside the seed vertex. If $o \geq 0$ the inferred read start lies inside v_{seed} and the pair (v_{seed}, o) is used directly. Otherwise a single backtracking step is performed over the immediate predecessors $p \in \text{pred}(v_{\text{seed}})$: for each predecessor whose label is long enough to absorb the negative offset, the start point $(p, |p.\mathit{label}| + o)$ is added to the candidate list, up to a fixed cap of 16 alternative origins. If no predecessor yields a valid offset, the aligner is invoked at $(v_{\text{seed}}, 0)$ as a fallback so that the read is still scored.

Window re-alignment: When a sequencing error falls near the start of a read, the seed-derived start offset can be displaced by a few bases relative to the position that minimises the affine-gap cost, so the aligner pays a spurious gap penalty and the reported coordinate drifts. To correct this, each candidate start (v_0, off_0) is expanded into a small window of nearby starts: for every shift $\delta \in [-w, +w]$ with $w = k$, the start $(v_0, \mathit{off}_0 + \delta)$ is generated, yielding $2w{+}1 = 11$ candidates per base origin (capped at 128 across all origins). Shifts below zero are resolved through the same predecessor walk used for negative seed offsets; shifts running past the end of a node are currently skipped. The shifts are tried in the order $0, -1, +1, -2, +2, \ldots$, so the most likely position is attempted first, and duplicate window starts are removed before the aligner is called. The lowest-scoring alignment over all starts is retained for that seed hit, and the best score over all regions and forks of the read is kept. This step was added because the seed-derived coordinate is an anchor, not necessarily the optimal affine-gap start position.

Theseus invocation: The aligner is invoked through a thin C-compatible wrapper declared in a header marked `extern "C"`. The single-start helper `run_theseus_alignment` delegates to `run_theseus_alignment_batch`, which receives the full list of window starts for one local subgraph. The wrapper constructs a `theseus::Graph` by iterating the node and edge arrays once, sanitises labels by stripping NUL, carriage-return and newline bytes that may leak in from input parsing, and maintains a hash map from GIN-TONIC vertex IDs to Theseus internal node IDs to translate every candidate start vertex. It instantiates a `TheseusAlignerImpl` with penalty parameters $(a, x, o, e) = (0, 5, 5, 1)$ for match, mismatch, gap-open and gap-extend, with heuristics disabled, and calls `aligner.align(read,` v_0`,` off_0`)` for each start in the batch. The score is computed via `compute_affine_gap_score`; the wrapper keeps the lowest-penalty valid alignment and exits early if a zero-cost alignment is found. The best-scoring result for the read (seed string, start vertex, start offset and penalty) is written to `alignment_log.txt`. GIN vertex IDs are zero-based internally; the log emits the external node coordinate by adding one, which is the convention used by the reference ground-truth coordinates in the evaluation.

D. Software Architecture

The pipeline is composed of three layers, each compiled independently and joined through narrow interfaces. The first layer is the existing GIN-TONIC C codebase, extended with the new `pigeonhole` mode in `gin.c` and with the auxiliary functions `extract_subgraph_bfs`, `build_theseus_data` and `gin_gin_verify_task` in `src/gin_gin.c`. The second layer is a header pair `wrapper/theseus_wrapper.{h,cpp}` that exposes the C-compatible `run_theseus_alignment` and

run_theseus_alignment_batch entry points and the matching POD types th_node_t, th_edge_t, th_start_t and theseus_result_t. The wrapper is the only translation unit that includes Theseus headers, isolating the GIN-TONIC C build from C++ symbol resolution. The third layer is Theseus itself, included under extern/theseus-lib. The top-level CMakeLists.txt adds the external dependency directory with add_subdirectory, links gin_lib_static against the theseus target, and exposes both extern/theseus-lib/include and the project header directory to the wrapper. Although the project-level C++ standard is set to C++20, the static GIN library advertises cxx_std_23 as a compile feature for the C++ wrapper path. The pigeonhole header is part of the normal header list so the CSR graph, region, task and timing structures are visible to both the command implementation and the verification code.

E. Instrumentation and Output

The current implementation records wall-clock timings with clock_gettime in a dedicated gin_pigeonhole_stats_t structure. The counters split execution into setup/loading, query reading, seeding, verification/alignment, and total runtime; per-read alignment log writes occur inside the verification phase, while output_time covers the final summary logging. At the end of the run the command appends a summary to alignment_log.txt, including total query time, cache build time, cache depth, number of processed queries, and the timing breakdown. This keeps the experimental measurements decoupled from the algorithmic code paths and gives the evaluation section a stable source for runtime data.

IV. Evaluation

A. Protocol

The evaluation uses an MHC graph containing 980 vertices and 1,399 directed edges, together with a pre-built GIN-TONIC index and depth-10 cache. All experiments run on an AMD Ryzen 7 7800X3D (8 cores, 16 threads). Two 10,000-read workloads are considered: short_match, made of exact simulated reads, and short_err, made of mutated reads generated from the same graph. The ground truth for each read provides the reference start position used to validate placements. The main runs use cache depth 10, error budget $k = 5$, occurrence threshold $\tau = 1$, and batch size 1024. Additional threshold-sweep runs use $\tau \in \{10, 30\}$ with the same inputs, cache, error budget, and batch size, but with $j \in \{1, 8\}$ for the threshold sweep. Runtime is measured from the normal execution logs with $j \in \{1, 4, 8\}$ threads for the main $\tau = 1$ run and with $j \in \{1, 8\}$ for the threshold sweep; memory is measured from the Valgrind/Massif profiles when available.

A read is counted as *aligned* when the command emits a [MIGLIORE] record. A placement is counted as an *exact start* when its reported start node and offset match the first mapping position in the ground truth. If the coordinate does not match that reference start, the evaluation performs a second conservative graph check on the MHC graph: starting from the reported coordinate, it accepts the placement only if the full read can be generated within the workload edit budget. Such cases are reported as *GFA-valid differences*: they disagree with the chosen ground-truth origin but are still valid alignments on the graph. The remaining emitted alignments are reported as *other differences*.

B. Summary

Table I is the main evaluation result. On exact reads, the pipeline emits 9,790 alignments. Of these, 9,782 start at the reference ground-truth coordinate, while the remaining eight start elsewhere but spell the read exactly on the GFA. Thus all emitted exact-read alignments are correct on the graph; the eight differences are alternative valid graph positions rather than invalid mappings. On mutated reads, the command emits an alignment for 98.15% of reads. Among the 338 starts that differ from the ground truth, 242 still pass the independent GFA edit-budget check, leaving 96 unresolved coordinate differences and giving a 99.02% correct-output rate among emitted alignments.

Tables II and III show the effect of relaxing the occurrence threshold. Increasing τ from 1 to 10 almost eliminates unaligned reads, reducing them from 210 to 8 on short_match and from 185 to 3 on short_err. Raising the threshold further to 30 leaves only one unaligned read in each workload. For exact reads, every coordinate difference introduced by the larger thresholds is again GFA-valid: 121/121 for $\tau = 10$ and 128/128 for $\tau = 30$. For mutated reads, the number of unresolved differences falls from 96 at $\tau = 1$ to 27 at $\tau = 10$ and 24 at $\tau = 30$. The extra sensitivity is therefore bought mainly by verifying many more candidate placements. At $j = 8$, exact-read runtime grows from 0.250 s at $\tau = 1$ to 13.657 s at $\tau = 10$ and 29.636 s at $\tau = 30$; mutated-read runtime grows from 5 s to 201 s and 454 s. The new single-thread logs show that the threshold-sweep runs retain similar parallel efficiency, between 93.7% and 94.5% at eight threads.

The important observation is that verification dominates the run. At $j = 8$, verification accounts for 92.5% of total time on exact reads and 99.6% on mutated reads. Seeding is therefore not the bottleneck on these workloads; most of the cost comes from constructing local candidate subgraphs and running Theseus on the generated start windows. The high parallel efficiency, 94.1% and 94.9% at eight threads, shows that the verification work is parallelisable across reads and candidate placements.

For the main $\tau = 1$ runs, the Massif profiles all report the same peak process footprint, 135.749 MB, for both workloads and all profiled thread counts. The peak consists of 127.029 MB of heap, 8.719 MB of Massif heap bookkeeping, and negligible stack usage in the summary. The absence of measurable growth from $j = 1$ to $j = 8$ indicates that the large structures (the index, cache, CSR graph, and decoder) are shared rather than replicated per worker. These profiles should be read as

TABLE I

END-TO-END EVALUATION SUMMARY. PERCENTAGES IN THE OUTCOME COLUMNS ARE COMPUTED OVER THE 10,000 PROCESSED READS IN EACH WORKLOAD, EXCEPT CORRECT OUTPUT, WHICH IS COMPUTED OVER EMITTED ALIGNMENTS. THE VERIFICATION SHARE IS REPORTED FOR THE EIGHT-THREAD RUN.

	Read outcome						Runtime				Memory
Dataset	Aligned	Exact start	GFA-valid diff.	Other diff.	Unaligned	Correct output (%)	$j = 1$ (s)	$j = 8$ (s)	Parallel eff. (%)	Verify (%)	Peak MB
short_match	9790	9782	8	0	210	100.00	1.882	0.250	94.1	92.5	135.749
short_err	9815	9477	242	96	185	99.02	37.301	4.912	94.9	99.6	135.749

TABLE II

END-TO-END EVALUATION SUMMARY FOR OCCURRENCE THRESHOLD $\tau = 10$. PERCENTAGES IN THE OUTCOME COLUMNS ARE COMPUTED OVER THE 10,000 PROCESSED READS IN EACH WORKLOAD, EXCEPT CORRECT OUTPUT, WHICH IS COMPUTED OVER EMITTED ALIGNMENTS. THE THRESHOLD SWEEP WAS RUN WITH $j = 1$ AND $j = 8$; MASSIF MEMORY WAS NOT PROFILED FOR THESE RUNS, SO MEMORY CELLS ARE MARKED WITH $--$.

	Read outcome						Runtime				Memory
Dataset	Aligned	Exact start	GFA-valid diff.	Other diff.	Unaligned	Correct output (%)	$j = 1$ (s)	$j = 8$ (s)	Parallel eff. (%)	Verify (%)	Peak MB
short_match	9992	9871	121	0	8	100.00	103.236	13.657	94.5	99.8	–
short_err	9997	9617	353	27	3	99.73	1510.464	200.893	94.0	100.0	–

TABLE III

END-TO-END EVALUATION SUMMARY FOR OCCURRENCE THRESHOLD $\tau = 30$. PERCENTAGES IN THE OUTCOME COLUMNS ARE COMPUTED OVER THE 10,000 PROCESSED READS IN EACH WORKLOAD, EXCEPT CORRECT OUTPUT, WHICH IS COMPUTED OVER EMITTED ALIGNMENTS. THE THRESHOLD SWEEP WAS RUN WITH $j = 1$ AND $j = 8$; MASSIF MEMORY WAS NOT PROFILED FOR THESE RUNS, SO MEMORY CELLS ARE MARKED WITH $--$.

	Read outcome						Runtime				Memory
Dataset	Aligned	Exact start	GFA-valid diff.	Other diff.	Unaligned	Correct output (%)	$j = 1$ (s)	$j = 8$ (s)	Parallel eff. (%)	Verify (%)	Peak MB
short_match	9999	9871	128	0	1	100.00	223.694	29.636	94.4	99.9	–
short_err	9999	9618	357	24	1	99.76	3406.499	454.519	93.7	100.0	–

Valgrind-instrumented peak footprints, not as native-runtime memory benchmarks.

All table values are computed from the execution logs, the ground-truth read origins, and the Massif summaries; the evaluation section embeds the table source directly rather than depending on auxiliary `.tex` fragments.

V. DISCUSSION AND LIMITATIONS

The experiments support the intended architecture. Exact-match discovery over the GIN-TONIC index is inexpensive, while local sequence-to-graph verification dominates end-to-end runtime. This is a useful outcome for the design: the seeding phase can be made more selective or more sensitive without being the main runtime bottleneck, and the verification work parallelises well because reads, seed hits, local subgraphs, and Theseus start windows are independent except for the final log write. The constant Massif peak across thread counts also indicates that the expensive structures (the index, cache, CSR graph, and decoder) are shared rather than replicated per worker.

The main accuracy limitation is not the affine-gap verifier, but candidate generation and coordinate recovery around the seed. The adaptive partition may emit fewer than $k+1$ regions when no sufficiently specific final region is found, weakening the strict pigeonhole sensitivity guarantee. A decoded seed whose implied read start falls before the seed node is back-tracked by at most one predecessor, and window shifts that pass the end of the current start node are skipped. These choices explain why a small fraction of reads remain unaligned or receive coordinates that do not exactly match the ground-truth origin.

Several refinements follow directly from these observations. A fallback fixed $k+1$ partition could be used when the adaptive thresholding loop would otherwise stop early, preserving worst-case sensitivity while keeping the selective adaptive seeds in the common case. Start recovery should be extended across multiple predecessor nodes and across rightward node boundaries inside the re-alignment window. Seed clustering and ranking would reduce repeated verification in repetitive graph regions, and the wrapper could expose CIGAR or path output from Theseus so that correctness can be assessed beyond the first reported coordinate. Finally, the command-line interface should make `-o` control the log path, and a complete mapper comparison should add baselines such as `vg` and GraphAligner on the same graph and reads.

VI. CONCLUSION

This project adds a complete pigeonhole-based seed-and-verify mode to GIN-TONIC. The implementation connects exact graph FM-index search, adaptive seed selection, CSR-based local graph extraction, batched Theseus verification, OpenMP parallel execution. On the MHC, the resulting prototype aligns roughly 98% of reads, validates all emitted exact-read alignments and 99.02% of emitted mutated-read alignments on the graph, and scales almost linearly to eight threads. The remaining gaps are well localised to sensitivity fallbacks, start-coordinate recovery, making the implementation a solid

foundation for a graph-native pigeonhole mapper rather than only a collection of isolated experiments.

REFERENCES

[1] T. Wang et al., "The Human Pangenome Project: a global resource to map genomic diversity," *Nature*, vol. 604, pp. 437–446, 2022.

[2] J. M. Eizenga et al., "Pangenome graphs," *Annual Review of Genomics and Human Genetics*, vol. 21, pp. 139–162, 2020.

[3] E. Garrison et al., "Variation graph toolkit improves read mapping by representing genetic variation in the reference," *Nature Biotechnology*, vol. 36, no. 9, pp. 875–879, 2018.

[4] J. Siren et al., "Haplotype-aware graph indexes," *Bioinformatics*, vol. 36, no. 2, pp. 400–407, 2020.

[5] S. Marco-Sola, M. Sammeth, R. Guigo, and P. Ribeca, "The GEM mapper: fast, accurate and versatile alignment by filtration," *Nature Methods*, vol. 9, no. 12, pp. 1185–1188, 2012.

[6] H. Li and R. Durbin, "Fast and accurate short read alignment with Burrows-Wheeler transform," *Bioinformatics*, vol. 25, no. 14, pp. 1754–1760, 2009.

[7] U. Ozturk, M. Mattavelli, and P. Ribeca, "GIN-TONIC: non-hierarchical full-text indexing for graph genomes," *NAR Genomics and Bioinformatics*, vol. 6, no. 4, lqae159, 2024.

[8] M. Rautiainen and T. Marschall, "GraphAligner: rapid and versatile sequence-to-graph alignment," *Genome Biology*, vol. 21, article 253, 2020.

[9] S. Marco-Sola, J. C. Moure, M. Moreto, and A. Espinosa, "Fast gap-affine pairwise alignment using the wavefront algorithm," *Bioinformatics*, vol. 37, no. 4, pp. 456–463, 2021.

[10] A. Jimenez et al., "Theseus-lib: fast, optimal affine-gap sequence-to-graph alignment library," GitHub repository, https://github.com/albertjimenezbl/theseus-lib.